1. PTAL calculation

PTAL uses official GTFS plus the Lombardia NeTEx supplement. For each grid cell, the workflow finds reachable public transport service access points and computes an Accessibility Index, `AI`.

PTAL formulas

Walking distance:

$$d_{ij}^{walk} = d_{ij}^{euclidean} \cdot c(d_{ij}^{euclidean})$$

where the circuity factor is:

$$c(d) = \begin{cases} 2.04, & d < 500 \\ 1.71, & 500 \leq d < 625 \\ 1.47, & 625 \leq d < 725 \\ 1.20, & d \geq 725 \end{cases}$$

Walking time:

$$WT_{ij} = \frac{d_{ij}^{walk}}{v \cdot 1000/60}$$

where `v` is walking speed in km/h.

Average waiting time:

$$AWT_{ij} = 0.5 \cdot \frac{60}{f_{ij}} + r_j$$

where:

$$f_{ij} = \text{service frequency per hour}$$

$$r_j = \text{mode reliability penalty in minutes}$$

Total access time:

$$TAT_{ij} = WT_{ij} + AWT_{ij}$$

Equivalent Doorstep Frequency:

$$EDF_{ij} = \frac{30}{TAT_{ij}}$$

Route weighting inside each mode:

$$wEDF_{ij} = \begin{cases} EDF_{ij}, & \text{strongest route in the mode} \\ 0.5 \cdot EDF_{ij}, & \text{additional routes in the same mode} \end{cases}$$

Accessibility Index:

$$AI_i = \sum_j wEDF_{ij}$$

PTAL class:

$$PTAL_i = \text{classify}(AI_i)$$

The class thresholds are:

$$PTAL_i = \begin{cases} 0, & AI_i \leq 0 \\ 1a, & 0 < AI_i \leq 2.5 \\ 1b, & 2.5 < AI_i \leq 5 \\ 2, & 5 < AI_i \leq 10 \\ 3, & 10 < AI_i \leq 15 \\ 4, & 15 < AI_i \leq 20 \\ 5, & 20 < AI_i \leq 25 \\ 6a, & 25 < AI_i \leq 40 \\ 6b, & AI_i > 40 \end{cases}$$

PTAL key source code

```
calc["walk_time_min"] = calc["walk_distance_m"] / (speed_kmh * 1000 / 60)
calc["average_wait_min"] = (
    0.5 * (60 / calc["freq_per_hour"]) + calc["reliability_min"]
)
calc["total_access_time_min"] = (
    calc["walk_time_min"] + calc["average_wait_min"]
)
calc["edf"] = 30 / calc["total_access_time_min"]

idx = calc.groupby(["grid_id", "mode", "service_key"])["edf"].idxmax()
dedup = calc.loc[idx].copy()
```

```
dedup["rank_in_mode"] = dedup.groupby(["grid_id", "mode"])["edf"].rank(
    method="first",
    ascending=False,
)
dedup["weighted_edf"] = np.where(
    dedup["rank_in_mode"] == 1,
    dedup["edf"],
    0.5 * dedup["edf"],
)

totals = (
    dedup.groupby("grid_id", as_index=False)
    .agg(
        ai=("weighted_edf", "sum"),
        accessible_services=("service_key", "nunique"),
        accessible_saps=("sap_id", "nunique"),
        accessible_operators=("operator", "nunique"),
        mean_wait_min=("average_wait_min", "mean"),
        mean_walk_min=("walk_time_min", "mean"),
    )
)
totals["ptal"] = totals["ai"].map(classify_ptal)
```

```
def classify_ptal(ai: float) -> str:
    if ai <= 0:
        return "0"
    if ai <= 2.5:
        return "1a"
    if ai <= 5:
        return "1b"
    if ai <= 10:
        return "2"
    if ai <= 15:
        return "3"
    if ai <= 20:
        return "4"
    if ai <= 25:
        return "5"
    if ai <= 40:
        return "6a"
    return "6b"
```

2. PTOL calculation

PTOL is a frequency-free public transport opportunity score. It counts how many transport modes are reachable on foot from each grid cell.

PTOL formulas

Walking time:

$$WT_{im} = \frac{d_{im}^{walk}}{v \cdot 1000/60}$$

Mode-specific reachability threshold:

$$\theta_m = \begin{cases} 8, & m \in \{\text{bus}, \text{tram}\} \\ 12, & m \in \{\text{metro}, \text{rail}\} \end{cases}$$

Mode access flag:

$$A_{im} = \begin{cases} 1, & WT_{im} \leq \theta_m \\ 0, & WT_{im} > \theta_m \end{cases}$$

Final PTOL:

$$PTOL_i = A_{i,\text{bus}} + A_{i,\text{tram}} + A_{i,\text{metro}} + A_{i,\text{rail}}$$

Therefore:

$$PTOL_i \in \{0, 1, 2, 3, 4\}$$

PTOL key source code

```
MODES = ["bus", "tram", "metro", "rail"]
MODE_THRESHOLDS_MIN = {
    "bus": 8.0,
    "tram": 8.0,
    "metro": 12.0,
    "rail": 12.0,
}
```

```
calc["walk_time_min"] = calc["walk_distance_m"] / (speed_kmh * 1000 / 60)
calc["threshold_min"] = calc["mode"].map(mode_threshold)
reachable = calc[calc["walk_time_min"] <= calc["threshold_min"]].copy()

grouped = (
    reachable.groupby(["grid_id", "mode"], as_index=False)
    .agg(
        min_walk_min=("walk_time_min", "min"),
        accessible_saps=("sap_id", "nunique"),
```

```
    )
)

for mode in MODES:
    mode_rows = grouped[grouped["mode"] == mode][
        ["grid_id", "min_walk_min", "accessible_saps"]
    ]
    result = result.merge(mode_rows, on="grid_id", how="left")
    result[f"{mode}_access"] = result["min_walk_min"].notna().astype(int)
    result[f"{mode}_min_walk_min"] = result["min_walk_min"]
    result[f"{mode}_accessible_saps"] = (
        result["accessible_saps"].fillna(0).astype(int)
    )

access_cols = [f"{mode}_access" for mode in MODES]
sap_cols = [f"{mode}_accessible_saps" for mode in MODES]
walk_cols = [f"{mode}_min_walk_min" for mode in MODES]

result["ptol"] = result[access_cols].sum(axis=1).astype(int)
result["ptol_accessible_saps"] = result[sap_cols].sum(axis=1).astype(int)
result["nearest_pt_stop_walk_min"] = result[walk_cols].min(axis=1, skipna=True)
```

3. PTD calculation

This is the report-based PTD calculation. It combines stop access, service frequency, line availability, hub access and a limited PTAL/PTOL component.

Normalization formulas

For positive accessibility variables:

$$x_i^{norm} = \frac{x_i - P_5(x)}{P_{95}(x) - P_5(x)}$$

$$x_i^{norm} = \min\left(1, \max\left(0, x_i^{norm}\right)\right)$$

For distance or time variables where lower is better:

$$x_i^{inv} = 1 - x_i^{norm}$$

PTD formulas

Stop access:

$$SA_i = 1 - \text{robust\_normalize}(nearest\_pt\_stop\_walk\_min_i)$$

Service frequency:

$$FR_i = \text{robust\_normalize}(total\_service\_frequency\_per\_hour_i)$$

Line availability:

$$LA_i = \text{robust\_normalize}(accessible\_line\_count_i)$$

Hub access:

$$HA_i = 1 - \text{robust\_normalize}(estimated\_walk\_time\_to\_nearest\_hub\_min_i)$$

PTAL component:

$$PTALcomponent_i = \text{robust\_normalize}(AI_i)$$

PTOL component:

$$PTOLcomponent_i = \frac{PTOL_i}{4}$$

Combined PTAL/PTOL component:

$$PTALPTOL_i = 0.5 \cdot PTALcomponent_i + 0.5 \cdot PTOLcomponent_i$$

Integrated Public Transport Accessibility:

$$PTA_i = 0.25SA_i + 0.30FR_i + 0.20LA_i + 0.15HA_i + 0.10PTALPTOL_i$$

Public Transport Deficit:

$$PTD_i = 1 - PTA_i$$

PTD key source code

```
def robust_minmax(series: pd.Series) -> pd.Series:
    numeric = pd.to_numeric(series, errors="coerce")
    valid = numeric.dropna()
    if valid.empty:
        return pd.Series(np.zeros(len(series)), index=series.index, dtype="float64")
    p5 = valid.quantile(0.05)
    p95 = valid.quantile(0.95)
    if not np.isfinite(p5) or not np.isfinite(p95) or abs(p95 - p5) < 1e-12:
        return pd.Series(np.zeros(len(series)), index=series.index, dtype="float64")
    normalized = (numeric - p5) / (p95 - p5)
    return pd.Series(clip01(normalized.fillna(0)), index=series.index, dtype="float64")

def robust_inverse_minmax(series: pd.Series) -> pd.Series:
    numeric = pd.to_numeric(series, errors="coerce")
    normalized = robust_minmax(numeric)
    inverse = 1 - normalized
    inverse[numeric.isna()] = 0
    return pd.Series(clip01(inverse), index=series.index, dtype="float64")

out["SA"] = robust_inverse_minmax(out["nearest_pt_stop_walk_min"])
out["FR"] = robust_minmax(out["total_service_frequency_per_hour"])
out["LA"] = robust_minmax(out["accessible_line_count"])
out["HA"] = robust_inverse_minmax(out["estimated_walk_time_to_nearest_hub_min"])
out["PTAL_component"] = robust_minmax(out["ai"])
out["PTOL_component"] = (
    pd.to_numeric(out["ptol"], errors="coerce").fillna(0).clip(0, 4) / 4.0
)
out["PTAL_PTOL"] = (
    (out["PTAL_component"] + out["PTOL_component"]) / 2
).clip(0, 1)
out["PTA"] = (
    0.25 * out["SA"]
    + 0.30 * out["FR"]
    + 0.20 * out["LA"]
    + 0.15 * out["HA"]
    + 0.10 * out["PTAL_PTOL"]
).clip(0, 1)
out["PTD"] = 1 - out["PTA"]
```

For the H3 WebGIS layer, 100 m PTD is aggregated to H3 mainly with GHSL population weighting:

$$PTD_h = \frac{\sum_{i \in h} PTD_i \cdot Pop_i}{\sum_{i \in h} Pop_i}$$

If population weights are unavailable, the script falls back to the simple mean.

```
record["PTD"] = weighted_mean(group, "PTD")
record["PTD_territorial_mean"] = float(
    pd.to_numeric(group["PTD"], errors="coerce").mean()
)
record["PTD_population_weighted"] = record["PTD"]
```